INF211

PROJECT-4

FOR STUDENTS

Project Description: Auto-Chess Battlegrounds

In this project you are asked to implement a series of Python's structures (class, class inheritance, dictionaries, functions etc.) that will enable you to create basic version of a card game depending on an randomness based auto-chess mechanism.

In means of keeping the track of what your program does in the background, you simply need to benefit from your object oriented programming skills to record the necessary data of the simulation (will be specified later).

Rules of the Game

- Game includes 6 players and a deck containing 30 cards.
- Cards in the deck represented by 2 different classes: character (18 cards) and spell (12 cards).
- Character cards having 2 symbolic stats as attack and health values. Spell cards carry either a damage or heal value.
- Players are assigned 1 special ability out of 3.
- Play mechanism is based on an Auto-chess logic and randomness mostly. So in a nutshell what your program must perform according to the given above is;

  Program must run 12 rounds, where each round, players are matched against to each other, each round 3 battle, players start the round with 8 health, specified deck is generated and players are distrubuted 5 cards each containing 2 spell and 3 character cards. Players can only play <u>1 card each turn</u> from their hands to field.
- After the players got matched and cards are dealt, matches start in order (not all at once but in an order) and players movements are written in a .txt file in the given format which will be provided below.
- According to a certain movement limit or until a player lost its all health, round must be resulted in 3 options,
  - Win: 3pts.
  - Draw: 1pts.
  - Lose: -1pts.
- In your code, as it's specified in the schemes below, you must also consider the options that user may want to add card or define a new player to the game, and when a card is added to the deck, <u>deck must be updated</u>.

Example .txt file snippets:

======== ROUND 1 ========

--- Match: Mira vs Kalen ---

Mira starting hand: [Thunder Rider (3/3), Frost Sentinel (2/4), Giant Shieldbearer (1/6), Ultimate Heal [heal 10], Meteor [damage 8], Ultimate Heal [heal 10]]

Kalen starting hand: [Arcane Duelist (3/2), Berserker (4/2), Stone Guardian (1/5), Holy Light [heal 3], Arcane Blast [damage 4]]

Mira played Ultimate Heal [heal 10]

Kalen played Berserker (4/2)

Kalen's Guardian Spirit directly hit Mira for 1 damage (player HP now 7)

…

…

Mira's Frost Sentinel attacked Kalen's Stone Guardian for 2 damage (remaining 3)

Kalen's Berserker attacked Mira's Frost Sentinel for 4 damage (remaining 0)

Kalen's Stone Guardian directly hit Mira for 1 damage (player HP now 1)

Kalen's Arcane Duelist directly hit Mira for 3 damage (player HP now -2)

Match Result: Kalen WINS

…

…

======== ROUND 12 ========

--- Match: Lyra vs Xera ---

Lyra starting hand: [Stone Guardian (1/5), Blade Adept (2/2), Battle Cleric (2/5), Fireball [damage 5], Holy Light [heal 3], Battle Cleric (2/5)]

Xera starting hand: [Mega Tank (1/15), Flame Soldier (3/1), Samurai Veteran (3/4), Frost Spear [damage 3], Ultimate Heal [heal 10], Arcane Bolt [damage 3]]

…

- As you can see from the example entries above, players' cards are dealt to their hands and played to the field in order to be able to use character cards. After a character cards is played to the field, or a spell at hand is decided to be played, movements written as entries can be specified as, attacks to either the opponent or it's minions, healing self or own field minions. <u>After each movement you must also clear the dead minions out of the battlefield</u>.

- Predefined deck and player names, abilities etc. is given below as, you may copy and paste it to your code freely.

```
character_data = [

    ("Rookie Warrior", 1, 3), ("Blade Adept", 2, 2), ("Stone Guardian", 1, 5),

    ("Arcane Duelist", 3, 2), ("Giant Shieldbearer", 1, 6), ("Forest Archer", 2, 3),

    ("Steel Knight", 3, 3), ("Berserker", 4, 2), ("Novice Monk", 1, 4),

    ("Flame Soldier", 3, 1), ("Frost Sentinel", 2, 4), ("Cave Brute", 4, 3),

    ("Samurai Veteran", 3, 4), ("Sand Assassin", 4, 1), ("Battle Cleric", 2, 5),

    ("Thunder Rider", 3, 3), ("Venom Hunter", 3, 2), ("High Paladin", 4, 4),

]


spell_data = [

    ("Fireball", "damage", 5), ("Heal", "heal", 4), ("Lightning", "damage", 7),

    ("Blessing", "heal", 6), ("Frost Spear", "damage", 3), ("Mend Wounds", "heal", 5),

    ("Meteor", "damage", 8), ("Rejuvenate", "heal", 7), ("Arcane Blast", "damage", 4),

    ("Holy Light", "heal", 3), ("Shadow Bolt", "damage", 6), ("Restoration", "heal", 5),

]
```

```
specials = [
    "extra_spell",
    "duplicate_card",
    "start_with_free_character",
    "extra_spell",
    "start_with_free_character",
    "duplicate_card"
]
names = ["Auron", "Shai", "Mediv" , "Xera", "Eanor", "Rauth"]
```

Note: Special abilities must do the followings,

- 'extra_spell' : Give the player's hand a predefined spell.

```
>>SpellCard("Arcane Bolt", "damage", 3)     # for example
```

- 'duplicate_card' : Give the player's hand another copy of a card from it's hand.

'start_with_free_character' : Give the player's field a predefined character.

```
>> CharacterCard("Guardian Spirit", 1, 3)     # for example
```

In order to make your job easier, you can follow the definitions for classes and methods below.

Class: Card()

Methods	Explanation
__init__(self,name)	An upper class to be inherited by character and spell classes, name will hold the cards name as str.

Class: CharacterCard(Card)

Methods	Explanation
__init__(self,name,attack,health) super().__init__(name)#inheritance	Will take character cards name, attack and health values as specified. chc = CharacterCard('Dummy','0','1')
__repr__(self):	When called, it must return the values it holds for assigned variable. >>chc.__repr__() 'Dummy (0/1)'

Class: SpellCard(Card)

Methods	Explanation
__init__(self, name, effect, value)	Will take spell cards name, effect (dmg/heal) and value as specified. sc = SpellCard('Whistle','heal',1)
__repr__(self)	When called, it must return the values it holds for assigned variable. >>sc.__repr__() 'Whistle [heal 1]'

Class: Deck()

Methods	Explanation
__init__(self)	Will hold the cards as a list of dictionaries as it is previously defined.
shuffle(self)	Will shuffle the list of dictionaries that represents cards.
draw(self)	Will choose a card from the deck and use appropriately in the called location. >>game.deck.draw() High Paladin (4/4)
copy_cards(self)	In order to keep the deck updated after adding a card, this method copies the current deck and deals to the players.

Class: Player()

Methods	Explanation
__init__(self, name, special)	Will hold the name and special given as inputs but also in order to play the game, it must hold a health value, cards in hand, cards in field and points it gained from the matches. p = Player('Anor','extra_spell') >> p.hand ['Berserker (3/3)','Flame (damage 4)']
clean_dead(self)	Will clean the player's field if a minion's health is below 0.
take_damage_on_field(self, amount, logf, attacker_player, attacker_card)	Needed for record and update the actions done in the Player's field. Amount represents the value that has been done by the character or spell card of opponents, logf to write the data to .txt file and

	attacker player and card are name's to fill the entry.
heal_field(self, amount, logf, healer_player, spell)	Same as taking damage, needed for updating the .txt file amount represents the value, healer player and spell holds the names.
play_card(self, card)	Simple move to take a card from hand to field.
apply_special(self)	According to the defined special ability, update the Players hand or field.
choose_card(self)	In order to increase randomness while playing a card from hand to field.

Class: Game()

Methods	**Explanation**
__init__(self)	Will hold the called classes; players and deck, also the logfile to record movements.
generate_deck(self)	Predefined character and spell cards (as list of dictionaries) will be assigned to the Deck() class here
view_deck(self)	Helper method to keep track of deck's current version.
view_players(self)	Helper method to keep track of current players.
add_new_card(self)	Method to test if your deck can be updated with a new card.
create_players(self)	Predefined Players (as list of strings) will be assigned to the Player() class here
add_new_player(self)	Method to test if your game can be played with a new player included.

deal_hands(self, temp_deck)	Method to deal 3 character and 2 spell card to each player randomly from the copied deck hold as 'temp_deck'.
resolve_spell(self, card, caster, target, logf)	Method to perform the spell card action to the cards on the field or to the opponent labelled as target and the user player as caster, card variable holds the spell's name and logf to record.
character_combat(self, p1, p2, logf)	Method to apply character card's (must be in the field) attack either to a enemy minion or directly to the opponent, logf to record the action.
fight(self, p1, p2, logf)	Method to perform hand to field transitions and resolve spell or combat for characters on the field until a player dies or a certain limit is reached. logf to keep record of actions.
run(self)	Method to initialize and open logf file. Also to copy the current deck, perform the all rounds and matches and card deals.

Note: After a character card played it can enter combat (attack to a opponent's minion or opponent) immediately but for once each turn.

if __name__ == "__main__":

You must do the necessary calls in here to perform the simulation and print the resulting points that players gathered.

-------- **GOOD LUCK!** ---------